Reinforcement Learning

Lecture 3: Reinforcement Learning Basics

SJTU

Outline

1. Recap and Motivation
2. Markov Decision Process
3. The RL Objective
4. Anatomy of an RL Algorithm
5. Value Functions and Q-Functions
6. Types of RL Algorithms
7. Summary

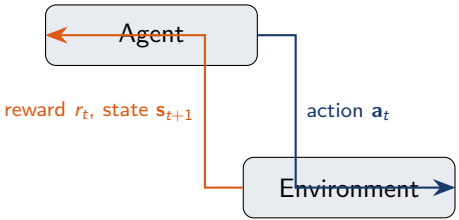
From Imitation to Reinforcement Learning

- Previous lectures: learning from **demonstrations** (imitation learning)
  - Behavioral cloning, DAgger — require expert data
- Limitation: expert demonstrations are **expensive** or **unavailable**
- Today: learn from **reward signals** instead
  - No expert needed — just a reward function
  - Agent discovers good behavior through **trial and error**

Reinforcement Learning: learn to maximize cumulative reward through interaction

Why Rewards?

- Rewards define **what** to achieve, not **how**
- Examples of reward functions:
  - Running: reward = forward velocity
  - Game playing: reward = score
  - Manipulation: reward = task completion
- Often easier to **specify** a reward than to **demonstrate** behavior
- Can discover **novel solutions** better than expert demonstrations



Markov Chain

Definition: Markov Chain

$\mathcal{M} = \{\mathcal{S}, \mathcal{T}\}$

- $\mathcal{S}$ — state space
- $\mathcal{T}$ — transition operator (matrix)



$$\mathcal{T} = \begin{pmatrix} p(s'_1|s_1) & p(s'_1|s_2) & \cdots \\ p(s'_2|s_1) & p(s'_2|s_2) & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

- **Markov property**:
  $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \ldots, \mathbf{s}_1) = p(\mathbf{s}_{t+1}|\mathbf{s}_t)$
- $\mathcal{T}_{i,j} = p(\mathbf{s}_{t+1} = i \mid \mathbf{s}_t = j)$
- Let $\boldsymbol{\mu}_{t,j} = p(\mathbf{s}_t = j)$, then:

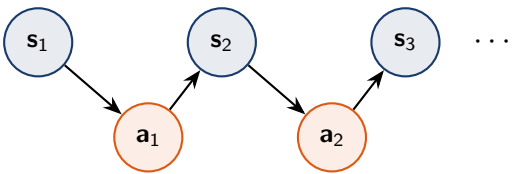
$$\vec{\mu}_{t+1} = \mathcal{T}\,\vec{\mu}_t$$

Markov Decision Process (MDP)

Definition: MDP

$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$

- $\mathcal{S}$ — state space
- $\mathcal{A}$ — action space
- $\mathcal{T}$ — transition operator: $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
- r — reward function: $r(\mathbf{s}_t, \mathbf{a}_t) \in \mathbb{R}$



MDP Transition Operator

- In an MDP, the transition operator acts on **state-action pairs**
- Let $\boldsymbol{\mu}_{t,j} = p(\mathbf{s}_t = j, \mathbf{a}_t = k)$
- The transition operator $\mathcal{T}$ is a tensor:

$$\mathcal{T}_{i,j,k} = p(\mathbf{s}_{t+1} = i \mid \mathbf{s}_t = j,\ \mathbf{a}_t = k)$$

- With policy $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, the **marginal** over states evolves as:

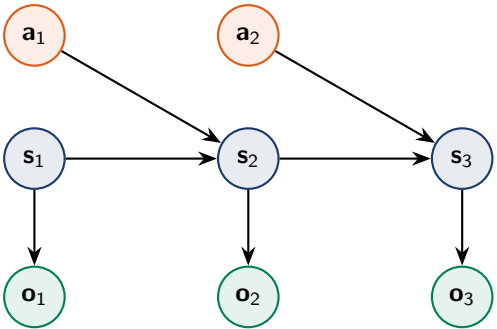
$$p(\mathbf{s}_{t+1} = i) = \sum_{j,k} \mathcal{T}_{i,j,k}\ \pi_\theta(\mathbf{a}_t = k \mid \mathbf{s}_t = j)\ p(\mathbf{s}_t = j)$$

Partially Observed MDP (POMDP)

Definition: POMDP

$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, r\}$

- $\mathcal{S}, \mathcal{A}, \mathcal{T}, r$ — same as MDP
- $\mathcal{O}$ — observation space
- $\mathcal{E}$ — emission probability: $p(\mathbf{o}_t|\mathbf{s}_t)$



Trajectory Distribution

- A **trajectory** $\tau = (\mathbf{s}_1, \mathbf{a}_1, \mathbf{s}_2, \mathbf{a}_2, \ldots, \mathbf{s}_T, \mathbf{a}_T)$
- The probability of a trajectory under policy π_θ:

$$p_\theta(\tau) = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)\, p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

- $p(\mathbf{s}_1)$ — initial state distribution
- $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ — policy (what we control)
- $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ — dynamics (given by environment)

This is a Markov chain on $(\mathbf{s}_t, \mathbf{a}_t)$ tuples with transition operator $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)\, \pi_\theta(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})$

State-Action Marginals

- From the trajectory distribution, we can extract **marginals**:

$$p_\theta(\mathbf{s}_t, \mathbf{a}_t) = \int p_\theta(\tau) \text{ keeping only } (\mathbf{s}_t, \mathbf{a}_t)$$

- The marginal tells us: *how likely is the agent to be in state* $\mathbf{s}_t$ *taking action* $\mathbf{a}_t$ *at time* t*?*
- Different from the **conditional** $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$:
  - The marginal depends on **all previous time steps**
  - The policy only depends on the **current state**

Finite Horizon RL Objective

- The goal of RL: find θ^* that maximizes **expected cumulative reward**

$$\theta^* = \arg\max_\theta \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

- Equivalently, using state-action marginals:

$$\theta^* = \arg\max_\theta \sum_{t=1}^{T} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim p_\theta(\mathbf{s}_t, \mathbf{a}_t)} \left[r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

- Note: the expectation depends on θ through the state-action marginals

Infinite Horizon: Stationary Distribution

- For $T \to \infty$: does $p_\theta(\mathbf{s}_t, \mathbf{a}_t)$ converge?
- Under regularity conditions (ergodicity), the Markov chain converges to a **stationary distribution** $\mu = \mathcal{T}\mu$
  - μ is the eigenvector of $\mathcal{T}$ with eigenvalue 1
  - Does not depend on initial state $p(\mathbf{s}_1)$
- Infinite horizon objective:

$$\theta^* = \arg\max_\theta \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim p_\theta(\mathbf{s}_t, \mathbf{a}_t)} \left[r(\mathbf{s}_t, \mathbf{a}_t) \right] \xrightarrow{T \to \infty} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \mu_\theta} \left[r(\mathbf{s}, \mathbf{a}) \right]$$

In practice, we often use a **discount factor** $\gamma < 1$ instead of the $1/T$ average.

Expectations and Smoothness

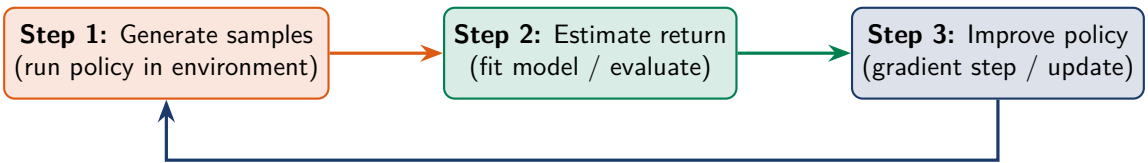
- Key insight: even if the reward $r(\mathbf{s}, \mathbf{a})$ is **discontinuous**...
- ...the RL objective $J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$ is **smooth** in θ
- Why? The expectation averages over all possible trajectories:

$$J(\theta) = \int r(\tau)\, p_\theta(\tau)\, d\tau$$

- If $p_\theta(\tau)$ is smooth in θ (e.g., Gaussian policy), then $J(\theta)$ is smooth
- This means we can use **gradient-based optimization**!

Expectations smooth out non-smooth rewards $\Rightarrow$ gradient methods work

The RL Algorithm Loop



- **Step 1**: Collect data $\{(\mathbf{s}_i, \mathbf{a}_i, r_i, \mathbf{s}'_i)\}$ by running π_θ in environment
- **Step 2**: Evaluate how good the collected trajectories are
- **Step 3**: Update θ to increase probability of good trajectories

Which Parts Are Expensive?

- **Step 1: Generate samples**
  - In simulation: usually **fast** (parallelizable)
  - In real world: **very expensive** (time, safety, wear)
- **Step 2: Estimate return / fit model**
  - Sometimes trivial (just sum up rewards)
  - Sometimes expensive (train a neural network model)
- **Step 3: Improve policy**
  - Usually a gradient step — **cheap** per iteration
  - But may need **many iterations** to converge

The cost balance between steps determines which RL algorithm to use.

Example: Policy Gradient (Preview)

1. **Generate samples**: run π_θ to collect trajectories $\tau^{(1)}, \ldots, \tau^{(N)}$
2. **Estimate return**: compute $R(\tau^{(i)}) = \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$
3. **Improve policy**: gradient ascent on

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) R(\tau^{(i)})$$

- This is the **REINFORCE** algorithm
- Direct: no value function, no model — just differentiate through the objective
- We will study this in detail in the next lecture

Example: Model-Based RL (Preview)

1. **Generate samples**: run π_θ to collect transitions $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$
2. **Fit model**: learn dynamics $f_\phi(\mathbf{s}, \mathbf{a}) \approx \mathbf{s}'$
3. **Improve policy**: use model to plan or backpropagate gradients

- Uses the model as a **learned simulator**
- Can be much more **sample efficient** than model-free methods
- But model errors can compound over long horizons

Model-based RL: learn the environment, then plan in the learned model

The Q-Function

Definition: Q-Function (Action-Value Function)

$$Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t, \mathbf{a}_t\right]$$

- $Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ = expected **total reward** from taking action $\mathbf{a}_t$ in state $\mathbf{s}_t$, then following policy π afterwards
- "How good is it to take this action in this state?"
- Depends on the policy π used for **future** actions

The Value Function

Definition: Value Function (State-Value Function)

$$V^{\pi}(\mathbf{s}_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\left[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t\right]$$

- $V^{\pi}(\mathbf{s}_t)$ = expected total reward from state $\mathbf{s}_t$ following policy π
- "How good is it to be in this state?"
- Relationship:

$$V^{\pi}(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}\left[Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)\right]$$

The RL objective $= \mathbb{E}_{\mathbf{s}_1 \sim p(\mathbf{s}_1)}\left[V^{\pi}(\mathbf{s}_1)\right]$

Using Q-Functions for Policy Improvement

- If we know $Q^\pi(\mathbf{s}, \mathbf{a})$, we can **improve** the policy:

$$\pi'(\mathbf{a}_t|\mathbf{s}_t) = \begin{cases} 1 & \text{if } \mathbf{a}_t = \arg\max_{\mathbf{a}} Q^\pi(\mathbf{s}_t, \mathbf{a}) \\ 0 & \text{otherwise} \end{cases}$$

- **Key insight**: if $\arg\max_{\mathbf{a}} Q^\pi(\mathbf{s}_t, \mathbf{a}) \neq \mathbb{E}[\mathbf{a}_t]$ under π, we can do better!
- This leads to **value-based** RL algorithms:
  1. Estimate Q^π or V^π from data
  2. Set $\pi' \leftarrow \arg\max Q^\pi$
  3. Repeat

The Advantage Function

Definition: Advantage Function

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - V^{\pi}(\mathbf{s}_t)$$

- $A^{\pi} > 0$: action $\mathbf{a}_t$ is **better than average** for state $\mathbf{s}_t$
- $A^{\pi} < 0$: action $\mathbf{a}_t$ is **worse than average** for state $\mathbf{s}_t$
- $A^{\pi} = 0$: action is exactly as good as following π
- The advantage tells us: "How much better is this specific action compared to what we would normally do?"

The advantage function is crucial for reducing variance in policy gradient methods.

Overview: Algorithm Families



- These are the four major families of RL algorithms
- They differ in:
  - What is **learned** (policy, value, model, or combinations)
  - How the policy is **extracted** (explicit vs. implicit)
  - **Sample efficiency** vs. **stability** tradeoffs

Policy Gradient Methods

- **Idea**: directly differentiate the RL objective $J(\theta)$ w.r.t. θ
- Gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- No value function needed (but can add one for variance reduction)

Examples

- **REINFORCE** — basic Monte Carlo policy gradient
- **PPO** (Proximal Policy Optimization) — clipped surrogate objective
- **TRPO** (Trust Region Policy Optimization) — constrained optimization

\+ Versatile, works with any differentiable policy

− High variance, typically on-policy (low sample efficiency)

Value-Based Methods

- **Idea**: learn $Q^\pi(\mathbf{s}, \mathbf{a})$ or $V^\pi(\mathbf{s})$, derive policy from it
- No explicit policy — just take $\arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$
- Fit Q-function using **Bellman equation** (bootstrapping)

Examples

- **Q-Learning** — tabular, off-policy
- **DQN** (Deep Q-Network) — neural network Q-function
- **Fitted Q-Iteration** — batch RL with function approximation

\+ Off-policy, sample efficient, low variance

− Typically discrete actions only, convergence not always guaranteed

Actor-Critic Methods

- **Idea**: combine policy gradient (actor) + value function (critic)
- **Actor**: policy $\pi_\theta(\mathbf{a}|\mathbf{s})$ — updated via policy gradient
- **Critic**: $V^\pi_\phi(\mathbf{s})$ or $Q^\pi_\phi(\mathbf{s}, \mathbf{a})$ — reduces variance of gradient estimate

Examples

- **A3C / A2C** — asynchronous advantage actor-critic
- **SAC** (Soft Actor-Critic) — maximum entropy framework
- **TD3** — twin delayed DDPG

\+ Lower variance than pure policy gradient, works with continuous actions

\− Two networks to train, more hyperparameters

Model-Based Methods

- **Idea**: learn transition dynamics $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$, then use the model
- Uses of the learned model:
  - **Planning**: search for good actions (e.g., Monte Carlo tree search)
  - **Backpropagation**: differentiate through the model to optimize policy
  - **Synthetic data**: generate imaginary rollouts for value function training

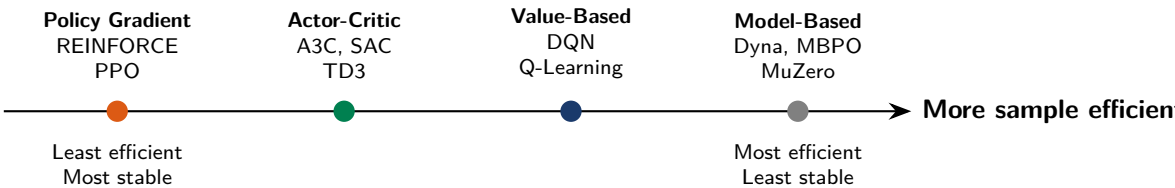
Examples

- **Dyna** — model-based with synthetic experience
- **MBPO** — model-based policy optimization
- **MuZero** — learned model + planning (AlphaGo family)
- **World Models** — learn latent dynamics

\+ Most sample efficient, enables planning

− Model errors compound, hard in high dimensions

Sample Efficiency Spectrum



- **Tradeoff**: more assumptions ⇒ more efficient ⇒ less robust
- No single best algorithm — depends on the problem!

On-Policy vs. Off-Policy

On-Policy

- Data collected by **current** policy π_θ
- Must generate **new** samples after each update
- Examples: REINFORCE, PPO, A3C

\+ Simpler, more stable

− Low sample efficiency

Off-Policy

- Can reuse data from **any** policy (replay buffer)
- One update per environment step possible
- Examples: DQN, SAC, TD3

\+ High sample efficiency

− Stability challenges

On-policy methods are easier to implement; off-policy methods are more practical for expensive environments.

Stochastic vs. Deterministic Policies

Stochastic Policy

$\pi_\theta(\mathbf{a}|\mathbf{s})$ is a **distribution**

- E.g., Gaussian: $\mathbf{a} \sim \mathcal{N}(\mu_\theta(\mathbf{s}), \sigma^2)$
- Natural exploration via randomness
- Required for policy gradient theorem
- Examples: PPO, SAC, A3C

Deterministic Policy

$\mathbf{a} = \mu_\theta(\mathbf{s})$ (single output)

- E.g., $\arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$
- Need separate exploration strategy
- Often used with value-based methods
- Examples: DQN, DDPG, TD3

Convergence and Stability

- **Policy gradient**: converges to local optimum with small enough step size
  - Gradient ascent on a (generally) non-concave objective
  - But the objective is smooth $\Rightarrow$ local optima are reachable
- **Value-based with function approximation**: no guarantees
  - The "deadly triad": function approximation + bootstrapping + off-policy
  - Can diverge in theory, but works well in practice with tricks
- **Model-based**: convergence depends on model accuracy
  - Model errors can compound and cause policy failure

Algorithm Comparison

Property	**Policy Grad.**	**Value-Based**	**Actor-Critic**	**Model-Based**
Learns	π_θ	Q / V	$\pi_\theta + V_\phi$	f_ϕ (dynamics)
Policy	explicit	$\arg\max Q$	explicit	varies
Action space	any	discrete	any	any
On/Off-policy	on	off	both	both
Sample eff.	low	medium	medium	high
Stability	high	medium	medium	low

These are general tendencies — specific algorithms may differ.

What We Will Cover in This Course

1. **Policy gradients**: REINFORCE, variance reduction, PPO, TRPO
2. **Actor-critic**: A2C/A3C, GAE, SAC
3. **Value-based**: Q-learning, DQN and extensions
4. **Model-based**: learned dynamics, planning, MuZero
5. **Advanced topics**: exploration, offline RL, multi-agent, RLHF

Summary

- **MDP**: $\{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$ — the formal framework for RL
  - POMDP adds observations $\mathcal{O}$ and emission $\mathcal{E}$
- **RL objective**: $\max_\theta \mathbb{E}_{\tau \sim p_\theta(\tau)}[\sum_t r(\mathbf{s}_t, \mathbf{a}_t)]$
  - Smooth in θ even for non-smooth rewards
  - Infinite horizon: stationary distribution
- **RL loop**: generate samples $\rightarrow$ estimate return $\rightarrow$ improve policy
- **Value functions**: $Q^\pi(\mathbf{s}, \mathbf{a})$, $V^\pi(\mathbf{s})$, $A^\pi(\mathbf{s}, \mathbf{a})$
  - Enable policy improvement via arg max
- **Algorithm families**: policy gradient, value-based, actor-critic, model-based
  - Tradeoff: sample efficiency vs. stability

Next lecture: Policy gradients — the REINFORCE algorithm and variance reduction

Thank You

Questions?